TUS LANDING_TRIAD_SET Specification v1.0

TUS OS®

Author: Mark Whitlock
Version: 1.0
Date: 13 March 2026

This specification defines the normative rules for TUS's active emitted Triad variant, `LANDING_TRIAD_SET_V1`: a deterministic three-start, three-pair construction that binds paired GUIDANCE and LANDING receipts into a conformant emitted set without changing the underlying kernel, core receipt machinery, or shared receipt semantics. It accompanies *The Triad System: Structural Triangulation for Verifiable AI Conformance*, which explains the architectural role of the Triad; this specification defines exactly how that role is realized in practice, including set construction, ordering, anchor mapping, deterministic LANDING behavior, emitted fields, and required set-level artifacts.

1. Authority

This companion specification SHALL be read together with the TUS core protocol.

This document is the separately versioned normative companion specification named in the TUS core protocol as `TUS LANDING_TRIAD_SET Specification v1.0`.

The TUS core protocol remains authoritative for shared receipt semantics, kernel behavior, receipt integrity, canonicalization, and general verifier obligations unless this document explicitly defines a Triad-variant rule.

This document is authoritative only for the variant-specific set behavior of `LANDING_TRIAD_SET_V1`.

It does not supersede or restate the full core protocol.

2. Purpose

This document defines the normative variant-specific behavior for the TUS Triad set variant `LANDING_TRIAD_SET_V1`.

It is a companion specification to the TUS core protocol. The core protocol remains authoritative for shared receipt semantics, kernel behavior, receipt integrity, canonicalization, and verifier behavior unless this document explicitly defines a variant-specific rule for the Triad set.

This document defines the conditions under which a Triad set is conforming, including how it is formed, emitted, identified, and interpreted at the set level.

3. Scope

This specification governs:

- Triad set construction from a single anchor start
- derivation of the two additional starts
- per-pair role assignment
- pair-to-anchor mapping for Triad LANDING
- deterministic post-anchor behavior inside the Triad variant
- required set-level artifacts
- set-level identity fields and binding behavior
- the distinction between canonical single-step selection and LANDING continuation endpoint semantics

This specification does not redefine:

- the TUS kernel
- CAST behavior in general
- GUIDANCE ranking rules
- receipt hashing rules
- canonicalization rules
- domain binding rules
- report-writing or narrative rendering rules
- general verifier obligations outside Triad-specific checks

4. Normative language

The key words MUST, MUST NOT, REQUIRED, SHALL, SHALL NOT, SHOULD, SHOULD NOT, and MAY in this document are to be interpreted as normative requirements.

5. Relationship to the core protocol

`LANDING_TRIAD_SET_V1` is a set-level orchestration over the existing fixed receipt machinery. It does not alter the underlying kernel, state-space rules, ranking logic, or core receipt integrity model.

The Triad variant adds a deterministic higher-order set operator that constructs three starts from one anchor and emits paired GUIDANCE and LANDING receipts for each of those starts.

The Triad therefore broadens the inspected conformance surface by widening starts, not by changing the receipt kernel.

6. Variant identity

A conforming Triad set under this specification SHALL identify itself with the following variant and rule identifiers:

- `set_variant = LANDING_TRIAD_SET_V1`
- `start_rule_id = START_SET_MAXIMIN_HAMMING_V1`
- `anchor_mapping_id = PAIR_INDEX_TO_ANCHOR_V1`

Where these identifiers are emitted at the set level, they MUST be present in the appropriate set-level artifacts and MUST be internally consistent across the emitted set.

For the active emitted v1 profile defined by this companion specification, the receipt-level mirror of `set_variant` is `variant`.

Where both fields are emitted, `set_variant` and `variant` SHALL carry the identical variant identifier value `LANDING_TRIAD_SET_V1`.

7. Set contract

A conforming Triad set SHALL contain exactly six receipt-bearing members arranged as three pairs.

Each pair SHALL contain:

- one GUIDANCE receipt
- one LANDING receipt

The pair members SHALL share the same `from_state`.

The set therefore has cardinality:

- `pair_count = 3`
- `receipts_per_pair = 2`
- `set_size = 6`

In this specification, `set_size` counts receipt-bearing members only. It does not count auxiliary audit surfaces or set-level bundle artifacts.

In the active emitted v1 profile defined by this companion specification, a conforming Triad bundle contains:

- 6 canonical receipt files
- 6 audit files paired to those receipts
- 4 set-level artifacts

Accordingly, the active emitted v1 bundle contains 16 emitted files in total, while `set_size` remains `6`.

For receipt-set binding and pairwise joinability, each emitted receipt-bearing member in the Triad set SHALL carry the following receipt-set join keys:

- `receipt_set.variant` (receipt-level mirror of `set_variant`)
- `receipt_set.set_id`
- `receipt_set.set_size`
- `receipt_set.pair_index`
- `receipt_set.pair_role`

- `start.start_role`

These join keys MUST be internally consistent across the emitted set and sufficient to recover set membership and pair membership without heuristic inference.

In the active emitted v1 profile defined by this companion specification, `start_role` SHALL be emitted on each canonical receipt-bearing member at the receipt path `start.start_role`.

That receipt-level field is the required local start-role witness for the individual receipt. For set-level pair-to-start mapping across the emitted Triad bundle, `triad.foundation` SHALL remain the authoritative rollup surface.

The three pairs SHALL be indexed as:

- Pair 1
- Pair 2
- Pair 3

Each pair SHALL also carry a start role:

- Pair 1 start role = `ANCHOR_A`
- Pair 2 start role = `DERIVED_B`
- Pair 3 start role = `DERIVED_C`

The Triad set contract defines pair structure and pair order, but it does not define a single universal receipt ordering across every emitted surface.

A conforming implementation and companion documentation MUST distinguish the following three orderings:

1. **Pair order**
    - Pair 1 uses `A`
    - Pair 2 uses `B`
    - Pair 3 uses `C`
2. **Bundle file order in the active emitted profile**
    - `01_Landing-1.canonical.json`
    - `02_Landing-1.audit.md`
    - `03_Guidance-1.canonical.json`
    - `04_Guidance-1.audit.md`
    - `05_Landing-2.canonical.json`
    - `06_Landing-2.audit.md`
    - `07_Guidance-2.canonical.json`
    - `08_Guidance-2.audit.md`
    - `09_Landing-3.canonical.json`
    - `10_Landing-3.audit.md`

    - `11_Guidance-3.canonical.json`
    - `12_Guidance-3.audit.md`
    - `13_triad.canonical.json`
    - `14_triad.foundation.json`
    - `15_triad.synthesis.json`
    - `16_manifest.json`
3. **`triad.canonical` member order and receipt-hash order in the active emitted profile**
    - `G1, F1, G2, F2, G3, F3`

These orderings are related but they are not interchangeable.

A conforming implementation MUST preserve the normative pair order. Where it emits the active profile, it SHALL preserve both the active bundle file order and the active `triad.canonical` member order in their respective surfaces.

Derived-start evidence is a set-level Triad coverage obligation, not a canonical per-receipt obligation.

In the active emitted v1 profile defined by this companion specification, the authoritative emitted home for that evidence SHALL be `triad.foundation` under the `starts` subtree.

That set-level evidence SHALL include, at minimum:

- the three emitted start identities for `A`, `B`, and `C` under `starts.*`
- `starts.distances.min_pairwise`
- `starts.distances.A_B`
- `starts.distances.A_C`
- `starts.distances.B_C`

These `triad.foundation` fields MUST be internally consistent with the active start set and sufficient to verify that the emitted derived starts conform to `START_SET_MAXIMIN_HAMMING_V1`.

If an implementation also emits convenience mirrors of that material on receipt-adjacent or audit-adjacent surfaces, those mirrors are non-authoritative for set-level derived-start verification unless a later emitted profile explicitly says otherwise.

8. Start construction

8.1 Anchor start

Triad construction begins from a single anchor start `A`.

`A` is the only non-derived start in the set.

The origin of `A` is outside the scope of this companion specification except that, for the active implementation profile, the Triad set is formed after anchor fixation. Once `A` is fixed, all remaining Triad behavior is deterministic.

8.2 Derived starts

Two additional starts, `B` and `C`, SHALL be derived deterministically from `A`.

Under the current TUS Core v1 authority surface, the operative fixed kernel dimension is `d = 6`.

Generic `d` notation in this companion specification is definitional only. It does not authorize alternate kernel dimensions under this version, and any different dimension requires a separately versioned protocol authority.

The derivation domain for fixed dimension `d` is the full legal TUS state space at that dimension. A conforming implementation MUST evaluate the derivation rule over that full legal state space and MUST NOT restrict the candidate domain to a subset, heuristic shortlist, sampled pool, or implementation-defined reduced search space.

The derivation rule is:

1. consider all legal distinct pairs `(B, C)` in the full legal state space for fixed dimension `d` such that `B != A`, `C != A`, and `B != C`
2. compute the minimum pairwise Hamming distance across the set `{A, B, C}`
3. choose the pair `(B, C)` that maximizes that minimum pairwise Hamming distance
4. if more than one pair is optimal, choose the lexicographically smallest ordered pair subject to `B < C`

This rule is identified by:

- `START_SET_MAXIMIN_HAMMING_V1`

8.3 Determinism requirement

For fixed dimension `d` and fixed anchor start `A`, the derived pair `(B, C)` MUST be unique under the rule above.

No stochastic selection, heuristic choice, or implementation latitude is permitted once `A` is fixed.

9. Pair ordering

The Triad set SHALL preserve the following ordering:

- Pair 1 uses start `A`
- Pair 2 uses start `B`
- Pair 3 uses start `C`

This ordering is not cosmetic. It is normatively significant because pair index governs the LANDING anchor mapping.

A conforming implementation MUST NOT permute pair order after derivation.

10. Pair roles

For each pair index `i in {1,2,3}` the implementation SHALL emit exactly two receipts:

- `GUIDANCE(i)`
- `LANDING(i)`

Both receipts in the pair MUST:

- originate from the same `from_state`
- agree on pair index
- agree on `start.start_role`
- agree on shared Triad set metadata where applicable

GUIDANCE and LANDING are therefore paired local witnesses over the same bounded start, not independent runs from different starts.

11. GUIDANCE behavior inside the Triad

GUIDANCE retains its normal receipt semantics inside the Triad variant.

For a fixed start state, GUIDANCE SHALL:

1. enumerate all legal single-line flips
2. compute the line ranking over that local neighborhood
3. select the top-ranked line `R[1]`
4. apply exactly one realized flip

Within the Triad variant, GUIDANCE therefore remains the dominant ranked local witness for each start.

This companion specification introduces no alternative GUIDANCE selection rule.

12. LANDING behavior inside the Triad

12.1 General rule

Inside `LANDING_TRIAD_SET_V1`, LANDING is not free-sampling LANDING.

Triad LANDING SHALL use anchored LANDING and SHALL NOT use sampled LANDING.

Once the anchor start `A` has been fixed and the Triad starts have been derived, Triad LANDING SHALL be deterministic.

The active LANDING rule for each pair is selected by pair index using the mapping defined in Section 13.

12.2 Prohibited stochastic behavior

A conforming Triad LANDING implementation MUST NOT perform RNG line sampling after Triad anchor fixation.

In particular, the Triad variant MUST NOT use stochastic landing selection in place of the required anchor mapping.

At the emitted artifact level, a conforming Triad LANDING profile MUST NOT contain:

- RNG line sampling for post-anchor LANDING line selection
- RNG trace entries for post-anchor LANDING line selection
- `LANDING_SAMPLE` reason codes for post-anchor LANDING selection

12.3 Anchored rule semantics

Triad LANDING selects a target rank index `k` from the pair mapping and applies the line occupying that rank at the pair start state.

For fixed start state and fixed pair index, the selected first-step LANDING line MUST therefore be unique.

13. Pair-to-anchor mapping

The pair-to-anchor mapping defined by this specification is identified by:

- `PAIR_INDEX_TO_ANCHOR_V1`

Under `PAIR_INDEX_TO_ANCHOR_V1`, each pair index is assigned a LANDING anchor identifier as follows:

- Pair 1 -> `LANDING_ANCHOR_RUNNER_UP`
- Pair 2 -> `LANDING_ANCHOR_MEDIAN`
- Pair 3 -> `LANDING_ANCHOR_LOWEST`

Each anchor identifier determines the selected rank index `k` as follows.

Under the current TUS Core v1 authority surface, `d = 6` for the active conforming kernel. The generic formulas below are definitional only and do not extend this specification to alternate dimensions:

- `LANDING_ANCHOR_RUNNER_UP` -> `k = 2`
- `LANDING_ANCHOR_MEDIAN` -> `k = ceil(d / 2)`
- `LANDING_ANCHOR_LOWEST` -> `k = d`

For dimension `d = 6`, the resulting selected ranks are therefore:

- Pair 1 -> `2`
- Pair 2 -> `3`
- Pair 3 -> `6`

A conforming implementation MUST emit pair-level metadata sufficient to show:

- pair index
- landing anchor identifier
- selected rank
- total ranks

14. Single-step and continuation semantics

Triad LANDING has two semantically distinct layers that MUST NOT be collapsed:

1. the canonical realized first-step transition
2. the internal continuation endpoint used for the Triad landing path

The first-step transition is the canonical realized move for receipt-valid local transition semantics.

The continuation endpoint is a separate internal second-step landing endpoint used by the Triad variant to represent the LANDING path under the same anchor index.

A conforming implementation, verifier profile, and companion documentation MUST keep these two meanings distinct.

A first-step transition MUST NOT be described as if it were the continuation endpoint.

A continuation endpoint MUST NOT be described as if it were the canonical realized first-step move.

15. First-step LANDING semantics

For Triad LANDING, the first step SHALL be determined as follows:

1. compute the ranked local neighborhood at the pair start state
2. determine the pair-specific anchor index `k`
3. select the line occupying rank `k`
4. apply exactly one realized flip on that line

This first realized flip is the canonical single-step LANDING move.

For conformance purposes, the emitted first-step move MUST remain identifiable as:

- the selected first-step LANDING line
- the realized first-step endpoint produced by that line

These first-step semantics are the authoritative local transition semantics for the LANDING receipt inside the Triad variant.

They MUST remain distinguishable from any later continuation state or continuation line.

16. Second-step LANDING continuation semantics

Where the active implementation emits a continuation step, the continuation step SHALL be derived deterministically from the first-step endpoint using the same anchor index `k`.

That is, the implementation SHALL:

1. recompute the local ranking at the first-step endpoint
2. select the line occupying rank `k` in that new local ranking
3. apply the second flip deterministically

For conformance purposes, the emitted continuation semantics MUST remain identifiable as:

- the selected continuation line
- the continuation endpoint produced by that line

This second step is not a replacement for the canonical first-step realized move.

It is a separate continuation endpoint in the Triad LANDING path.

A conforming implementation MUST NOT merge first-step and continuation semantics into a single undifferentiated endpoint meaning.

17. Emitted receipt semantics for Triad LANDING

In the active emitted implementation profile, a conforming Triad LANDING receipt MUST carry first-step and continuation semantics in distinct emitted fields.

For the first step:

- the selected first-step LANDING line SHALL be emitted as `chosen_line_step1`
- in the active emitted v1 profile, the selected first-step LANDING line SHALL also be mirrored in `selection.chosen_line`
- the canonical realized first-step endpoint SHALL be emitted through `selection.to_state_hex` and `selection.to_state_id` in the active emitted v1 profile

For the continuation step:

- the selected continuation line SHALL be emitted as `chosen_line_step2`
- the continuation endpoint SHALL be emitted as `landing_to_state_hex` and `landing_to_state_id` in the active emitted v1 profile

The active emitted profile SHALL also emit pair-specific LANDING metadata sufficient to identify the anchored rule instance, including:

- `landing_anchor.anchor_id`
- `rank_selected`
- `total_ranks`
- `triad_anchor_trace`

Where `triad_anchor_trace` is emitted in the active profile, it SHALL carry at least the following leaf fields:

- `anchor_id`
- `k_target_rank`
- `selected_line`
- `selected_line_rank`

These emitted fields are not interchangeable.

`chosen_line_step1`, `selection.to_state_hex`, and `selection.to_state_id` identify the canonical realized first-step LANDING move.

`chosen_line_step2`, `landing_to_state_hex`, and `landing_to_state_id` identify the separate continuation semantics of the Triad LANDING path.

A conforming implementation, validator, or companion documentation MUST NOT treat `landing_to_state_hex` / `landing_to_state_id` as if they were the canonical first-step endpoint, and MUST NOT treat `chosen_line_step2` as if it were the realized first-step line.

The `selection.chosen_line` mirror required above is specific to the active emitted v1 profile defined by this companion specification. It is not, by itself, the abstract minimum requirement for every possible future emitted profile.

18. Required set-level artifacts

A conforming emitted Triad bundle SHALL include the following set-level artifacts:

- `triad.canonical`
- `triad.foundation`
- `triad.synthesis`
- `manifest`

The abstract conformance requirement is the presence of these four set-level artifacts with their required semantic roles.

For the active v1 emitted profile defined by this companion specification, these artifacts SHALL appear as the following numbered bundle files:

- `13_triad.canonical.json`
- `14_triad.foundation.json`
- `15_triad.synthesis.json`
- `16_manifest.json`

Within this specification, those numbered filenames are not optional profile examples. They are the required emitted filenames for the active v1 profile.

A future implementation MAY define a different emitted filename surface only under a separately identified emitted profile or later specification revision. That possibility is outside the scope of this document.

The semantic roles remain defined by artifact type and binding function, but for the active v1 emitted profile those roles are carried by the numbered filenames above.

19. Artifact roles

19.1 triad.canonical

`triad.canonical` is the set-level identity and membership artifact.

It SHALL bind:

- set variant
- set id
- anchor start identity

- start rule identity
- anchor mapping identity
- ordered member list
- member receipt hashes
- member pair roles and start roles

In the active emitted profile, the ordered member list and corresponding receipt-hash order SHALL be:

- `G1, F1, G2, F2, G3, F3`

`triad.canonical` is authoritative for set membership and set-level identity.

19.2 triad.foundation

`triad.foundation` is the structured fact artifact.

It SHALL provide grounded extracted facts from the emitted set without adding recommendations, speculative interpretations, or unsupported state claims.

All summaries, extracted facts, and structured statements in `triad.foundation` MUST remain grounded in emitted receipt or set-level fields.

`triad.foundation` MUST NOT introduce invented identifiers, inferred states, inferred transitions, or interpretive conclusions that are not directly supported by emitted receipt or set-level material.

`triad.foundation` MAY summarize starts, pair roles, selected ranks, chosen lines, and endpoint identities only where those summaries remain fully grounded in emitted receipt or set-level fields.

For the active emitted v1 profile defined by this companion specification, `triad.foundation` SHALL also be the authoritative set-level home for derived-start coverage evidence.

At minimum, it SHALL carry:

- the `starts` coverage subtree for the active Triad start set
- `starts.distances.min_pairwise`
- `starts.distances.A_B`
- `starts.distances.A_C`
- `starts.distances.B_C`

Within this companion specification, those `triad.foundation` leaves are the authoritative emitted surfaces for verifying start-set separation under `START_SET_MAXIMIN_HAMMING_V1`.

19.3 triad.synthesis

`triad.synthesis` is the bounded interpretive artifact.

It is downstream from `triad.foundation` and MUST remain bound to it.

In the active emitted implementation profile, `triad.synthesis.derived_from.foundation_hash` SHALL bind to `triad.foundation.integrity.canonical_hash`.

A conforming implementation, validator, or companion documentation MUST NOT reinterpret `foundation_hash` as the raw SHA-256 of the foundation file bytes.

Raw file-byte SHA-256 binding for `triad.foundation.json` belongs to manifest integrity, not to synthesis binding semantics.

It MUST NOT introduce:

- new states
- new transitions
- recommendations
- unsupported causal or predictive claims

19.4 manifest

`manifest` is the file-packaging and transport-integrity artifact.

In the active emitted v1 profile, it SHALL bind file names, byte counts, and file hashes for the emitted bundle members other than `16_manifest.json` itself, unless a later emitted profile explicitly defines manifest self-inclusion.

Any such bindings MUST be internally consistent with the active emitted bundle.

20. Reporting role asymmetry

Within the Triad reporting model, LANDING is the primary report-facing surface and GUIDANCE is supporting structural evidence.

This asymmetry affects reporting emphasis only.

Report-facing priority does not create additional receipt authority for LANDING beyond the authority already carried by the emitted paired receipts.

It does not alter the underlying receipt authority of the paired runs, and it MUST NOT be interpreted as changing kernel semantics or receipt validity rules.

21. Conformance requirements

A conforming Triad set MUST satisfy at least the following:

1. exactly three starts and six paired receipts
2. each GUIDANCE/LANDING pair shares the same `from_state`
3. starts ordered as `A`, `B`, `C`
4. pair order is preserved and not permuted after derivation
5. `B` and `C` derived from `A` under `START_SET_MAXIMIN_HAMMING_V1`
6. pair-to-anchor mapping under `PAIR_INDEX_TO_ANCHOR_V1`
7. GUIDANCE uses `R[1]` at each start
8. LANDING first step uses the pair-selected rank index `k`

9. no stochastic LANDING selection after anchor fixation
10. required `triad.foundation` start-set coverage evidence is present and internally consistent
11. set-level identifiers are internally consistent across emitted artifacts
12. set-level artifact hashes and manifest bindings are internally consistent
13. synthesis remains bounded to foundation

22. Non-goals

This specification does not claim that the Triad performs global search, optimal planning, causal discovery, or unconstrained scenario generation.

The Triad is a bounded conformance mechanism built from paired local witnesses over three maximally separated starts.